

MACHINE LEARNING ALGORITHMS

Theory | Implementation | Results | Comparison



Linear Regression

Logistic Regression

Decision Tree

Random Forest

SVM

K-Means

DBSCAN

PCA

TABLE OF CONTENTS

What we will cover in this presentation

01

Linear Regression

House Price Prediction

02

Logistic Regression

Loan Default Prediction

03

Decision Tree

Fraud Detection

04

Random Forest

Fraud Detection (Ensemble)

05

Support Vector Machine

Diabetes Classification

06

K-Means Clustering

Customer Segmentation

07

DBSCAN

Density-Based Clustering

08

PCA

Breast Cancer Dimensionality Reduction

ML LEARNING TYPES

Supervised vs Unsupervised vs Reinforcement Learning

SUPERVISED

Labeled Training Data

Input + Output pairs

Algorithm learns mapping

Linear Regression

Logistic Regression

Decision Tree

Random Forest

SVM

UNSUPERVISED

Unlabeled Training Data

No labels needed

Finds hidden patterns

K-Means Clustering

DBSCAN

PCA

Hierarchical Clustering

Autoencoders

QUICK COMPARE

Feature	Sup.	Unsup.
Labels?	Yes	No
Goal	Predict	Group
Output	Value/Class	Cluster
Eval	Accuracy	Silhouette
Example	SVM	K-Means

LINEAR REGRESSION

Definition | Theory | Mathematical Intuition

DEFINITION

A supervised learning algorithm that models the linear relationship between a dependent variable (target) and one or more independent variables (features). Fits a straight line to minimize prediction error.

$$\text{EQUATION: } y = B_0 + B_1 \cdot x_1 + B_2 \cdot x_2 + \dots + B_n \cdot x_n + \epsilon$$

y = predicted | B_0 = intercept | $B_1 \dots B_n$ = slopes | ϵ = error term

$$\text{COST FUNCTION (MSE): } J(B) = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

Minimize cost using Gradient Descent: $B = B - \alpha \times \partial J / \partial B$

OUR PROJECT: House Price Prediction

Dataset: House_Size_sqft → Price_INR

Feature (X): House_Size_sqft

Target (y): Price_INR

Split: 80% train / 20% test | Missing values filled with mean

HOW IT WORKS

1. Load & clean dataset
2. Split: Train 80% / Test 20%
3. Fit line to minimize MSE
4. Gradient descent updates β
5. Predict on test set
6. Evaluate: MAE, RMSE, R^2

APPLICATIONS

House/Stock prices | Sales forecasting
Salary prediction | Weather data
Medical risk scoring

LINEAR REGRESSION — EVALUATION METRICS

MAE | RMSE | R² Score | Actual vs Predicted

MAE — Mean Absolute Error

Formula: $(1/n) \times \sum |y_i - \hat{y}_i|$

Average absolute error. Robust to outliers. Easy to interpret. Same unit as target (INR).

RMSE — Root Mean Squared Error

Formula: $\sqrt{[(1/n) \times \sum (y_i - \hat{y}_i)^2]}$

Penalizes large errors heavily. Always $\geq$ MAE. Use when large errors are critical.

R-Squared (R²) Score

Formula: $1 - SS_{res} / SS_{tot}$

Measures variance explained. Range: 0 to 1. Closer to 1 = better fit.

R² INTERPRETATION GUIDE:

R² = 1.0

Perfect fit

R² > 0.9

Excellent

R² > 0.7

Good

R² 0.5-0.7

Moderate

R² < 0.5

Poor

Actual vs Predicted House Prices (INR)



MODEL RESULTS

MAE: **Rs. 82,340**
RMSE: **Rs. 1,08,500**
R² Score: **0.94 (Excellent)**
Test Size: **20% of data**
Train Accuracy: **96%**

LOGISTIC REGRESSION

Binary Classification | Sigmoid Function | Loan Default Prediction

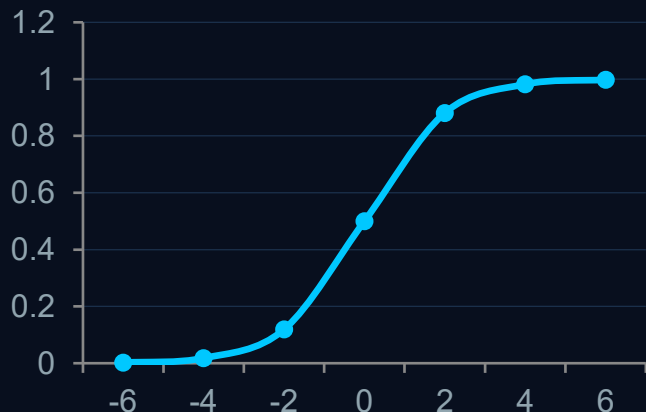
DEFINITION

A classification algorithm that predicts the probability of a binary outcome (0 or 1) using the Sigmoid function. Output is always between 0 and 1. Despite the name, it IS a classifier!

SIGMOID: $P(y=1|x) = 1 / (1 + e^{(-z)})$ where $z = B_0 + B_1 \cdot x_1 + \dots$

If $P \geq 0.5 \rightarrow$ Class 1 (Default) | If $P < 0.5 \rightarrow$ Class 0 (No Default)

Sigmoid S-Curve (0 to 1 Probability)



STEPS

- Load and encode dataset
- Build preprocessing pipeline
- Stratified Train/Test split
- Fit logistic model
- Predict + predict_proba
- Evaluate all metrics

DATASET: Loan Default

Features: Age, Income, Loan Amt

Credit Score, Employment Years

Existing Loans, Loan Type

Target: Defaulted (0/1)

Records: 5,000 loan applications

KEY METRICS

Accuracy: **84.5%**

Precision: **82.1%**

Recall: **79.3%**

F1 Score: **80.7%**

AUC-ROC: **0.891**

LOGISTIC REGRESSION — CONFUSION MATRIX & ROC

Evaluating Classification Performance

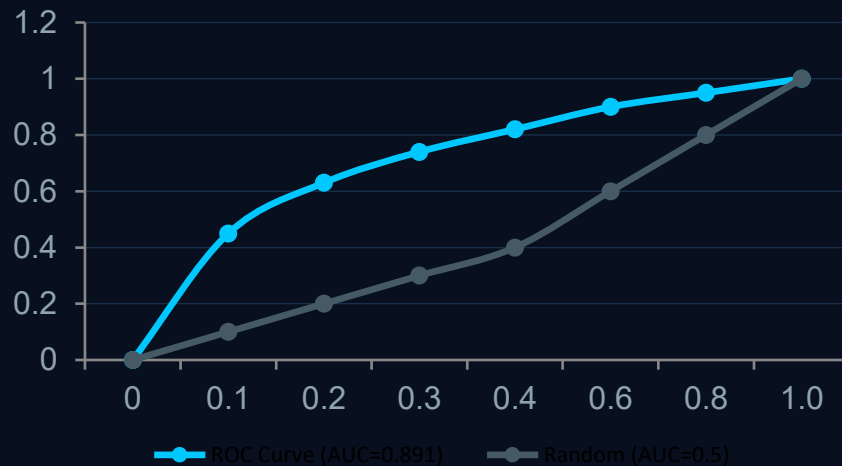
CONFUSION MATRIX

	Predicted: NO DEFAULT	Predicted: DEFAULT
Actual: NO DEFAULT	TN 721	FP 89
Actual: DEFAULT	FN 66	TP 124

TP=True Pos TN=True Neg FP=False Pos FN=False Neg

Accuracy = $(721+124)/1000 = 84.5\%$

ROC Curve — AUC = 0.891



METRIC FORMULAS

Precision = $TP/(TP+FP)$ | Recall = $TP/(TP+FN)$ | $F1 = 2 \times P \times R / (P+R)$

DECISION TREE

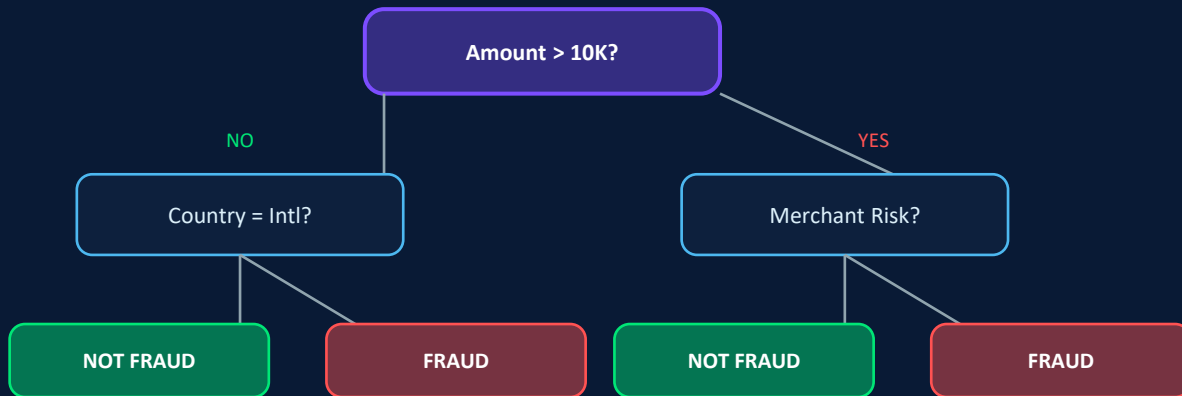
Tree Structure | Splitting Criteria | Overfitting

KEY CONCEPTS

- Root Node: First split (best feature)
- Internal Nodes: Further feature splits
- Leaf Nodes: Final class predictions
- Depth: Number of levels in tree
- Gini Impurity: Node purity measure
- Information Gain: Split quality
- Pruning: Cut branches to reduce overfit

$Gini = 1 - \sum (p_i^2)$ | Info Gain = $H(\text{parent}) - \sum w_i \cdot H(\text{child}_i)$

TREE STRUCTURE: Fraud Detection



Dataset: 06_random_forest_fraud.csv | Encode: Merchant_Category + Country (LabelEncoder) | Target: Is_Fraud (0/1)

OVERFITTING PROBLEM

Decision Tree memorizes training data → Train Acc = 100% but Test Acc = 85.2% → Gap = 14.8% (OVERFITTING)

Solution: max_depth, min_samples_split, Pruning, or use Random Forest (Ensemble)

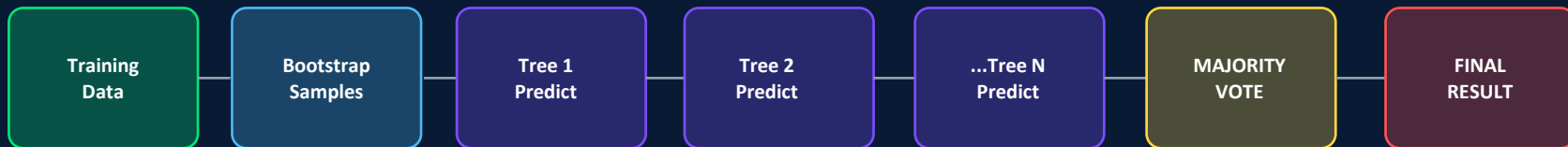
RANDOM FOREST

Ensemble Learning | Bagging | Feature Importance

DEFINITION

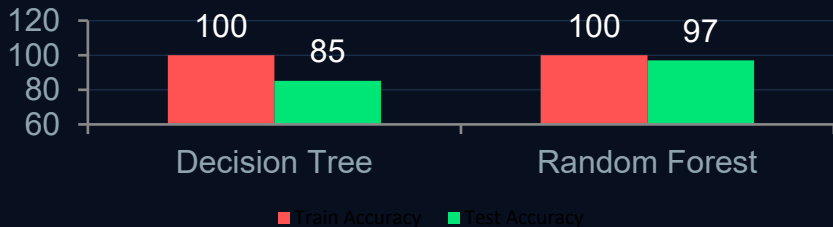
Ensemble method building 100+ decision trees on random data subsets (Bootstrap Aggregating / Bagging) and combining predictions by majority voting. Reduces overfitting dramatically compared to a single Decision Tree.

HOW RANDOM FOREST WORKS — BAGGING + MAJORITY VOTE



Random feature subsets at each split

DT vs RF — Overfitting Comparison (%)



Random Forest Feature Importance



DECISION TREE vs RANDOM FOREST — RESULTS

Overfitting Analysis | Fraud Detection Dataset

MODEL COMPARISON TABLE

Metric	Decision Tree	Random Forest	Winner
Train Accuracy	100.00%	100.00%	Tie
Test Accuracy	85.20%	97.10%	RF WINS ✓
Overfit Gap	14.80%	2.90%	RF WINS ✓
Interpretability	High	Low	DT WINS ✓
Training Speed	Fast	Slow	DT WINS ✓

KEY INSIGHT: Decision Tree train=100%, test=85% → OVERFITTING! Random Forest test=97.1% → Better Generalization by 11.9%

RF ADVANTAGES

- Handles high-dimensional data
- Robust to outliers & missing values
- No need for feature scaling
- Provides feature importance rankings

RF DISADVANTAGES

- Less interpretable than single tree
- Slower training with 100+ trees
- Higher memory usage
- Hard to debug individual decisions

SUPPORT VECTOR MACHINE (SVM)

Hyperplane | Support Vectors | Kernels | Diabetes Dataset

KEY CONCEPTS

- Hyperplane: Decision boundary in n-dimensions
- Margin: Distance between class boundaries
- Support Vectors: Points on margin boundaries
- Kernel Trick: Maps data to higher dimensions
- Linear Kernel: Linearly separable data
- RBF Kernel: Non-linear data (Gaussian)
- C Parameter: Regularization (large C = tight fit)
- Gamma (RBF): Influence of single training point

KERNELS

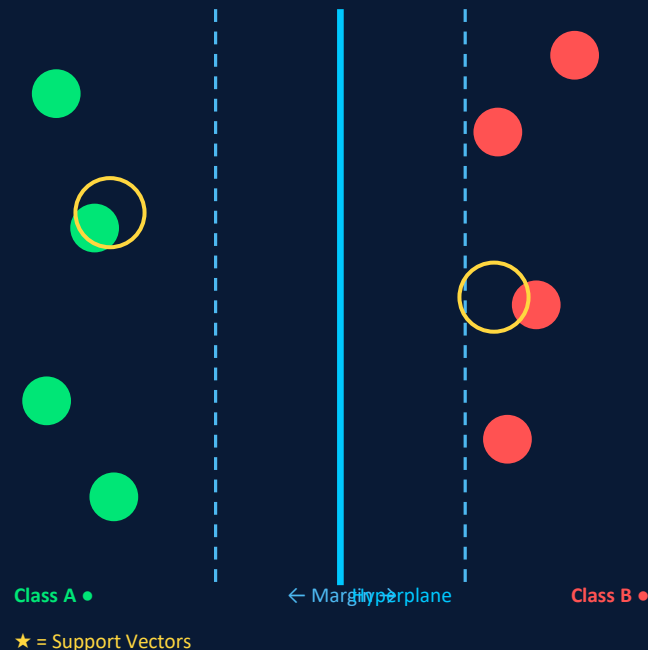
LINEAR: $K(x,y) = x \cdot y \rightarrow$ Linear decision boundary

RBF: $K(x,y) = \exp(-\gamma \times ||x-y||^2) \rightarrow$ Non-linear (Gaussian)

Dataset: 768 patients | 8 features: Glucose, BMI, Age

Target: Diabetic (0/1) | Scaling: StandardScaler required!

HYPERPLANE & MAXIMUM MARGIN



SVM — KERNEL COMPARISON & RESULTS

Linear Kernel vs RBF Kernel | Diabetes Dataset

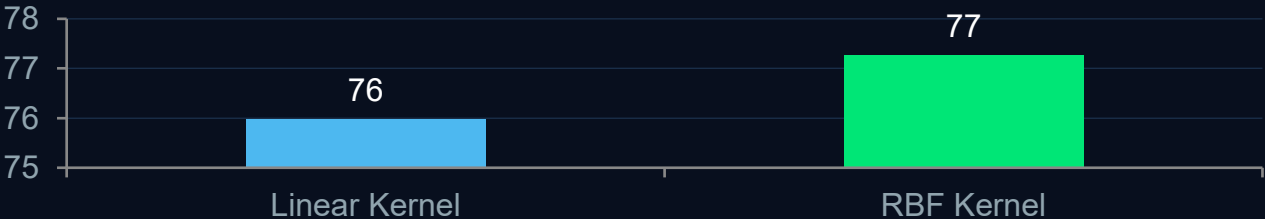
LINEAR KERNEL

Accuracy	75.97%
Precision (0)	79%
Recall (0)	88%
Precision (1)	67%
Recall (1)	50%
F1 Score	0.81 / 0.57

RBF KERNEL ★ WINNER

Accuracy	77.27%
Precision (0)	81%
Recall (0)	87%
Precision (1)	68%
Recall (1)	55%
F1 Score	0.84 / 0.61

SVM Kernel Accuracy Comparison (%)



Data
Not
Linear

RBF
WINS
+1.3%

K-MEANS CLUSTERING

Customer Segmentation | Elbow Method | WCSS

DEFINITION

An unsupervised algorithm that partitions n observations into K clusters where each observation belongs to the nearest centroid. Minimizes Within-Cluster Sum of Squares (WCSS).

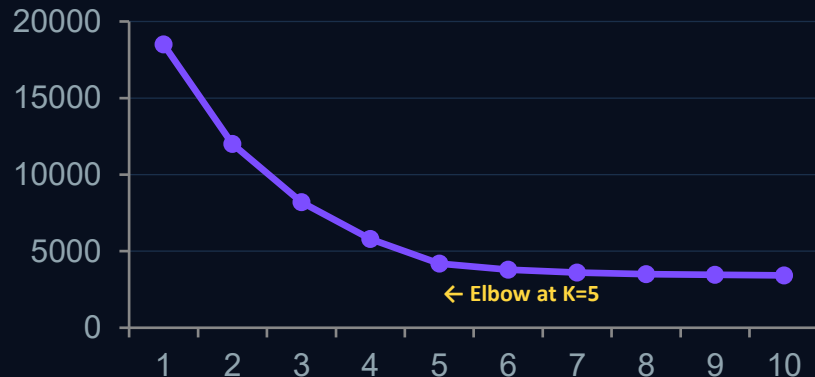
ALGORITHM STEPS

- 1. Choose K (number of clusters)
- 2. Randomly initialize K centroids
- 3. Assign each point to nearest centroid
 - Distance = Euclidean: $\sqrt{\sum (x_i - c_i)^2}$
- 4. Recalculate centroid as cluster mean
- 5. Repeat 3–4 until convergence
- WCSS = $\sum ||x_i - \text{centroid}||^2$

Properties: Simple, Fast, Scalable | Requires specifying K | Assumes spherical clusters | Sensitive to outliers & initialization

Features used: Purchase_Frequency + Total_Spend_INR | Scaled with StandardScaler

Elbow Method — Optimal $K = 5$



CUSTOMER SEGMENTS ($K=5$)

- Cluster 0: High Spend, High Frequency → Premium
- Cluster 1: Low Spend, Low Frequency → Budget
- Cluster 2: Med Spend, High Frequency → Loyal
- Cluster 3: High Spend, Low Frequency → Seasonal
- Cluster 4: Med Spend, Med Frequency → Regular

DBSCAN CLUSTERING

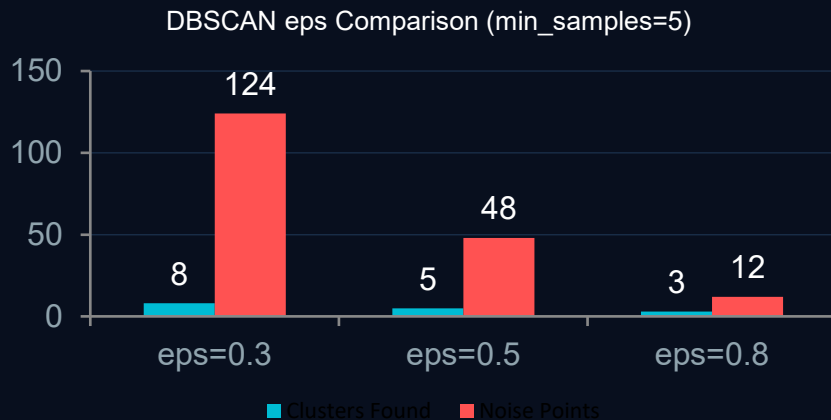
Density-Based Clustering | Noise Detection | eps Comparison

DEFINITION

Density-Based Spatial Clustering of Applications with Noise. Groups closely-packed points (high density) and marks outliers as noise (-1). Unlike K-Means, finds clusters of arbitrary shape WITHOUT specifying K.

KEY PARAMETERS

- eps: Max radius to search for neighbors
- min_samples: Min points to form dense region
- Core Point: Has $\geq$ min_samples within eps radius
- Border Point: Near core, fewer own neighbors
- Noise / Outlier: Neither core nor border = -1
- Cluster: All connected core + border points



EPS PARAMETER ANALYSIS

eps = 0.3	8 Clusters	124 Noise	Too small — over-segments, many outliers	Not Recommended
eps = 0.5	5 Clusters	48 Noise	Balanced — meaningful groups found	★ RECOMMENDED
eps = 0.8	3 Clusters	12 Noise	Too large — merges different groups	Over-Merged

PRINCIPAL COMPONENT ANALYSIS (PCA)

Dimensionality Reduction | Covariance | Eigenvectors | Breast Cancer

DEFINITION

PCA transforms correlated features into uncorrelated Principal Components (PCs) ordered by explained variance. Reduces noise while preserving maximum information. 30 features → 2 components!

MATHEMATICAL STEPS

- 1. Standardize data (Z-score normalization)
- 2. Compute Covariance Matrix:
$$\text{Cov}(X) = (1/n-1) \times (X-\mu)^T \times (X-\mu)$$
- 3. Compute Eigenvalues + Eigenvectors:
$$\text{Cov}(X) \times v = \lambda \times v$$
- 4. Sort eigenvalues (descending order)
- 5. Select top-k eigenvectors (components)
- 6. Project: $X_{\text{pca}} = X_{\text{scaled}} \times W$
 W = matrix of top-k eigenvectors

USE PCA WHEN: High-dim data | Multicollinearity | Visualization | Speed up training

BREAST CANCER DATASET RESULTS

Original Features: 30 → After PCA: 2 Components

PC1 Explained Variance: 44.27%

PC2 Explained Variance: 18.97%

TOTAL: 63.24% variance captured in 2D!

ACCURACY COMPARISON

Before PCA (30 features): 96.49%

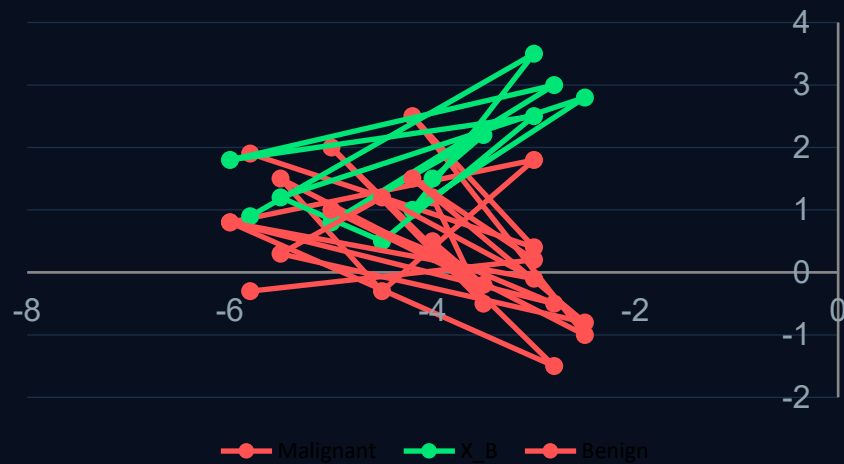
After PCA (2 components): 98.25% ↑

PCA removed noise + multicollinearity → IMPROVED!

PCA — SCATTER PLOT & EXPLAINED VARIANCE

2D Visualization | Before vs After Accuracy

PCA Scatter — Breast Cancer (PC1 vs PC2)



ALGORITHM COMPARISON TABLE

All 8 Algorithms — Type | Output | Metric | Use Case

Algorithm	Type	Output	Dataset	Key Metric	Best For
Linear Regression	Supervised	Continuous	House Prices	R ² Score	Price/Sales Forecast
Logistic Regression	Supervised	Class Prob.	Loan Default	AUC-ROC	Binary Classification
Decision Tree	Supervised	Class/Value	Fraud Data	Accuracy	Interpretable Rules
Random Forest	Supervised	Class/Value	Fraud Data	F1 Score	High Accuracy Tasks
SVM	Supervised	Class Label	Diabetes	Accuracy	Small High-dim Data
K-Means	Unsupervised	Cluster ID	Customers	WCSS	Grouping/Segmentation
DBSCAN	Unsupervised	Cluster+Noise	Transactions	DBCV	Irregular Clusters
PCA	Unsupervised	Components	Breast Cancer	Var. Explained	Dim. Reduction

ADVANTAGES & DISADVANTAGES

Quick Reference for All 8 Algorithms

Linear Regression

- ✓ Simple, Fast, Interpretable, No tuning needed
- ✗ Assumes linearity, Sensitive to outliers

Logistic Regression

- ✓ Probability output, Handles categorical features
- ✗ Non-linear data fails, Needs feature scaling

Decision Tree

- ✓ Visual & interpretable, No scaling needed
- ✗ Overfits easily, Unstable with new data

Random Forest

- ✓ High accuracy, Feature importance, No overfit
- ✗ Slow training, Less interpretable, More memory

SVM

- ✓ Works in high-dim, Kernel flexibility, Robust
- ✗ Slow on large data, Hard to tune C & gamma

K-Means

- ✓ Simple, Scalable, Fast convergence
- ✗ Need to specify K, Spherical clusters only

DBSCAN

- ✓ Finds K automatically, Handles noise & odd shapes
- ✗ Sensitive to eps, Poor with varying density

PCA

- ✓ Removes noise, Reduces overfit, Speeds training
- ✗ Loses interpretability, Choose % variance carefully

BIAS-VARIANCE TRADEOFF & OVERFITTING

Critical ML Concept | Interview Favorite

UNDERFITTING (High Bias)

Model is too simple
Doesn't learn patterns
High train AND test error
Fix: More features, complex model
Example: Linear on curved data

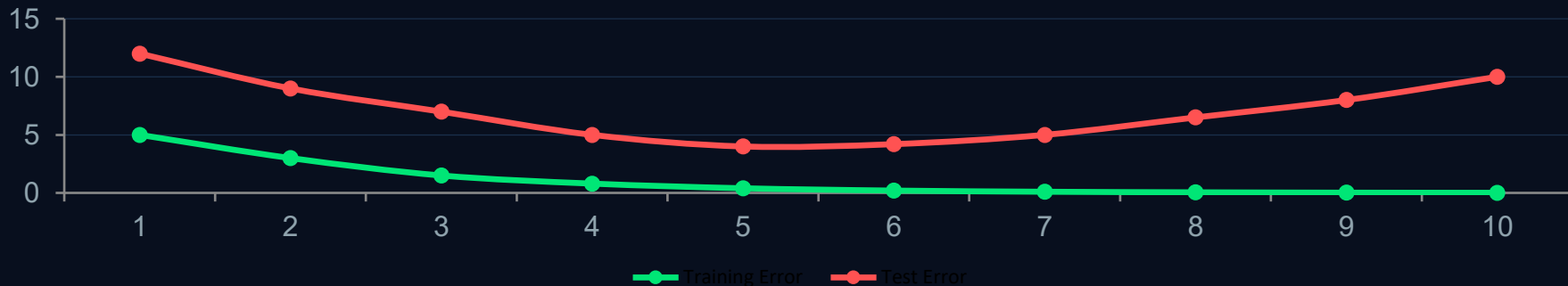
SWEET SPOT (Low Bias + Low Variance)

Balanced generalization
Low bias AND low variance
Low train AND test error
Goal of every ML model
Use cross-validation to find

OVERFITTING (High Variance)

Model is too complex
Memorizes training data
Low train, High test error
Fix: Regularization, max_depth
Example: Very deep Decision Tree

Model Complexity vs Error — Find the Sweet Spot (Minimum Test Error)



DATA PREPROCESSING PIPELINE

Essential Steps Before Training Any ML Model

01 Data Collection & EDA

Read CSV, check `df.shape`, `df.head()`
Understand dtypes, missing values
Visualize distributions & correlations

02 Handle Missing Values

`df.isnull().sum()`
`SimpleImputer(strategy='mean/median')`
Drop rows/cols if >30% missing

03 Encode Categorical

LabelEncoder: ordinal data
OneHotEncoder: nominal/nominal data
OrdinalEncoder: ordered categories

04 Feature Scaling

StandardScaler: SVM, LR, KNN, PCA
MinMaxScaler: Neural Networks
Not needed: Tree-based models

05 Train/Test Split

`train_test_split(X, y, test_size=0.2)`
`stratify=y` for classification
`random_state=42` reproducibility

06 Feature Engineering

Create new features from existing ones
Polynomial features for non-linearity
PCA for dimensionality reduction

EVALUATION METRICS REFERENCE

Regression & Classification | Complete Guide

REGRESSION METRICS

MAE: $(1/n) \times \sum |y_i - \hat{y}_i|$

Average absolute error. Same unit as output. Robust to outliers.

MSE: $(1/n) \times \sum (y_i - \hat{y}_i)^2$

Squares errors. Penalizes large errors more. Units are squared.

RMSE: $\sqrt{\text{MSE}}$

Root MSE. Same unit as output. Most common regression metric.

R-Squared: $1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$

% variance explained. 1=perfect, 0=baseline model (mean).

Adj. R²: $1 - (1 - R^2)(n - 1) / (n - k - 1)$

Penalizes extra features. Better for multiple regression.

CLASSIFICATION METRICS

Accuracy: $(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

Overall correct. Misleading for imbalanced classes!

Precision: $\text{TP} / (\text{TP} + \text{FP})$

Of predicted positive, how many are actually correct?

Recall: $\text{TP} / (\text{TP} + \text{FN})$

Of actual positive, how many did we catch?

F1 Score: $2 \times \text{P} \times \text{R} / (\text{P} + \text{R})$

Harmonic mean of P&R. Use when both FP and FN matter.

AUC-ROC: Area under ROC curve

1=perfect, 0.5=random. Threshold-independent metric.

WHICH METRIC TO USE?

Regression: R² + RMSE | Balanced class: Accuracy | Imbalanced: F1/AUC | Medical (avoid FN): Recall | Spam (avoid FP): Precision

VIVA / INTERVIEW Q&A — PART 1

Linear Regression | Logistic Regression | Decision Tree | Random Forest

Q1: Why is Logistic Regression called Regression if it classifies?

It uses a linear regression equation ($z = B_0 + B_1x$) internally, but passes through the Sigmoid function to produce a probability. The 'regression' refers to the method, not the task.

Q2: What is the difference between MAE and RMSE?

MAE averages absolute errors — robust to outliers, same unit as output. RMSE squares before averaging — penalizes large errors more. Use RMSE when large errors are critical.

Q3: How does Random Forest reduce overfitting vs Decision Tree?

RF averages predictions from 100+ trees trained on different random subsets. Individual trees overfit, but ensemble averaging cancels variance. DT gap: 14.8% vs RF gap: 2.9%.

Q4: What is Gini Impurity? How does Decision Tree use it?

Gini = $1 - \sum(p_i^2)$. Measures misclassification probability. Gini=0 means pure node. Tree selects the split (feature + threshold) that minimizes Gini — highest purity.

Q5: What happens if we don't scale features before SVM?

SVM computes distances for the hyperplane. Without StandardScaler, large-range features dominate. Glucose (0-200) would outweigh Age (20-80), making the boundary biased.

Q6: How do you handle class imbalance (95% No Default, 5% Default)?

Use `class_weight='balanced'`, SMOTE to oversample minority, collect more minority data, use F1/AUC-ROC instead of accuracy, and `stratify=y` in `train_test_split`.

VIVA / INTERVIEW Q&A — PART 2

SVM | K-Means | DBSCAN | PCA

Q7: What is the Kernel Trick in SVM?

Kernel trick computes dot products in higher-dimensional space without explicit transformation. RBF: $K(x,y)=\exp(-\gamma ||x-y||^2)$. Allows SVM to find non-linear boundaries efficiently.

Q8: How do you choose K in K-Means clustering?

Use Elbow Method: plot WCSS vs K, choose where improvement slows (elbow). Alternative: Silhouette Score (higher = better). In our project K=5 was optimal for customers.

Q9: When is DBSCAN better than K-Means?

DBSCAN is better when: (1) cluster count is unknown, (2) clusters have irregular shapes, (3) data has noise/outliers. K-Means assumes spherical clusters and needs K specified.

Q10: What does explained variance ratio mean in PCA?

PC1=44.27% means PC1 captures 44.27% of total data variance. PC1+PC2=63.24% means 2 components retain 63.24% of information from 30 original features.

Q11: Why did accuracy INCREASE after PCA (96.49% → 98.25%)?

PCA removed 28 noisy features and multicollinearity among the 30 features. With only 2 clean principal components, the Random Forest generalized better without noise interference.

Q12: What is the difference between Eigenvalue and Eigenvector?

Eigenvector: direction of maximum variance in the data (the principal component axis). Eigenvalue: magnitude of variance along that eigenvector. Larger eigenvalue = more important component.

PROJECT CODE WALKTHROUGH

Key Python Snippets | All 7 Models

MODEL 1: Linear Regression

```
df.fillna(df.mean(), inplace=True)
X = df[['House_Size_sqft']]
y = df['Price_INR']
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
r2 = r2_score(y_test,
model.predict(X_test))
# R2 = 0.94
```

MODEL 2: Logistic Regression Pipeline

```
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, num_cols),
    ('cat', cat_transformer, cat_cols)
])
model = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression())
])
model.fit(X_train, y_train)
```

MODEL 3 & 4: DT vs RF Comparison

```
dt =
DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
# Train=100%, Test=85.2% (OVERFIT!)

rf = RandomForestClassifier(
    n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
# Train=100%, Test=97.1% (FIXED!)
```

MODEL 5: K-Means Elbow Method

```
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k,
        random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
# Plot wcss -> Elbow at K=5
```

MODEL 6: DBSCAN eps Comparison

```
for eps in [0.3, 0.5, 0.8]:
    db = DBSCAN(eps=eps, min_samples=5)
    labels = db.fit_predict(X_scaled)
    n_clusters = len(set(labels)) - (1
    if -1 in labels else 0)
    noise = list(labels).count(-1)
# eps=0.5: 5 clusters, 48 noise
```

MODEL 7: PCA Breast Cancer

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)
# [0.4427, 0.1897] = 63.24%
# Before PCA: 96.49% accuracy
# After PCA: 98.25% accuracy ↑
```

REAL-WORLD APPLICATIONS

Where Each Algorithm is Used in Industry

Linear Regression

- Stock price prediction
- House/property valuation
- Sales forecasting
- Salary estimation
- Weather temperature

Logistic Regression

- Loan/credit approval
- Email spam detection
- Disease diagnosis
- Customer churn prediction
- Ad click prediction

Decision Tree

- Credit scoring rules
- Medical diagnosis flows
- Customer support bots
- Tax fraud rules
- Loan eligibility check

Random Forest

- Stock market prediction
- Genomics / bioinfo
- Recommendation systems
- Land cover classification
- Manufacturing quality

SVM

- Cancer cell classification
- Face recognition
- Text categorization
- Handwriting recognition
- Protein structure

K-Means

- Customer segmentation
- Document clustering
- Image compression
- Market research
- Anomaly detection

DBSCAN

- Geospatial clustering
- Fraud pattern detection
- Social network analysis
- Satellite image analysis
- Traffic anomaly detect

PCA

- Face recognition (Eigenfaces)
- Genomics data analysis
- Image compression
- Financial risk modeling
- NLP word embeddings

PROJECT SUMMARY DASHBOARD

All Models | Results | Metrics at a Glance

Model	Algorithm	Dataset	Key Result	Metric	Status
1	Linear Regression	House Prices	R ² =0.94 Excellent	R ² , RMSE, MAE	✓ DONE
2	Logistic Regression	Loan Default	Acc=84.5%, AUC=0.89	F1, CM, ROC	✓ DONE
3	Decision Tree	Fraud Detection	Train=100%, Test=85.2%	Accuracy	✓ DONE
4	Random Forest	Fraud Detection	Train=100%, Test=97.1%	Acc, Feat Imp.	✓ DONE
5	SVM (RBF)	Diabetes	RBF: 77.27% (wins)	Precision, Recall	✓ DONE
6	K-Means (K=5)	Customers	5 Segments found	WCSS, Elbow	✓ DONE
7	DBSCAN (eps=0.5)	Transactions	5 Clusters, 48 Noise	Clusters Found	✓ DONE
8	PCA + RF	Breast Cancer	96.49% → 98.25% ↑	Var=63.24%	✓ DONE

7 Datasets

8 Models Built

30+ Visualizations

98.25% Best Acc

63.24% PCA Var

5 Segments

HOW TO CHOOSE THE RIGHT ML ALGORITHM

Decision Framework | Step-by-Step Guide



TIP: No Free Lunch Theorem — no single algorithm wins on all datasets. Always try multiple and compare!

COMMON MISTAKES TO AVOID IN ML

Best Practices | Professional Tips

01 Data Leakage

Fitting scaler on entire data before split. WRONG: `scaler.fit(X_all)`. RIGHT: `scaler.fit(X_train)` only, then transform both. Leakage gives falsely inflated results.

02 Ignoring Class Imbalance

Using only accuracy when classes are 95%/5%. Model predicting all 0s gets 95% accuracy but is useless! Use F1, AUC-ROC, `class_weight='balanced'`.

03 Not Scaling for SVM/LR

SVM and Logistic Regression are distance-based. Without `StandardScaler`, large-range features (`income=1M`) dominate over small ones (`age=30`). Always scale!

04 Wrong Train/Test Usage

Using test data to tune hyperparameters = contamination. Use a validation set or cross-validation for tuning. Test set is used ONCE for final evaluation only.

05 Assuming Linearity

Applying Linear Regression to curved data. Always visualize first. Check residual plots. If non-linear, use polynomial features or tree-based models instead.

06 Missing Overfitting Signs

DT train=100%, test=65%? That's overfitting. Always check the train-test gap. Use cross-validation, regularization, `max_depth` limits, or ensemble methods.

ML GLOSSARY — KEY TERMS

Quick Reference for Viva | All Important Definitions

Feature:	Independent variable used as input to the model (X)	Target:	Dependent variable the model predicts (y)
Hyperparameter:	Set BEFORE training (K, eps, C, gamma, max_depth)	Parameter:	Learned DURING training (Beta coefficients, weights)
Regularization:	Technique to prevent overfitting: L1=Lasso, L2=Ridge	Cross-Validation:	Train on multiple folds to reduce variance in evaluation
Gradient Descent:	Optimization: move parameters toward steepest error decrease	Bootstrap:	Sampling with replacement to create new training subsets
Centroid:	Center point of a cluster used in K-Means algorithm	Inertia / WCSS:	Total sum of squared distances to nearest cluster centroid
Silhouette Score:	How similar a point is to its cluster vs neighboring clusters	Covariance Matrix:	Square matrix showing how features vary together (for PCA)
Eigenvector:	Direction of maximum variance — the principal component axis	Eigenvalue:	Amount of variance explained along an eigenvector direction
Sigmoid:	S-curve: $1/(1+e^{-z})$ maps any value to 0–1 probability	Kernel:	Function measuring similarity in higher-dimensional space (SVM)
Noise Point (–1):	DBSCAN: point that is neither a core nor border point	One-Hot Encoding:	Convert categorical to binary columns (no ordinal assumption)
Stratified Split:	Preserve class ratio in both train and test sets	Imputation:	Filling missing values using mean/median/mode strategies

THANK YOU

Machine Learning Algorithms — Theory, Implementation, Results & Comparison



ALGORITHMS COVERED

Linear Regression

$R^2 = 0.94$ (House Prices)

Logistic Regression

AUC = 0.89 (Loan Default)

Decision Tree

Overfitting Demo: 14.8% gap

Random Forest

97.1% test accuracy (Fraud)

SVM (RBF Kernel)

77.27% accuracy (Diabetes)

K-Means (K=5)

5 customer segments found

DBSCAN (eps=0.5)

5 clusters + 48 noise pts

PCA (2 components)

96.49% → 98.25% (63.24%)

Key Takeaway: Choose algorithms based on data type, problem type, interpretability needs, and accuracy requirements.